Project description

Part 1 – Hedge Funds Performance Tool: project and database

By analyzing the 2020 hedge fund database, this programme provides insights for prospective investors, based on funds' performance in terms of returns, as well as their utility (location). A subset of the external database "Top 100 Global Hedge Funds 2020" by Shashank Srivastava, comprising 33 hedge funds, was used to support this programme*. Each dataset entry includes the name of the hedge fund, its city and country of location, assets under management, and annual returns.

Project functionalities

The programme offers five main options for analyzing the hedge fund database:

Option 1 (Calculate the Average Return for Each Country): This option calls the **average_returns_by_country** function, which computes the average annual returns for hedge funds grouped by their country of location. The process involves utilizing the **sum_returns** function, which calculates the total returns using recursion. These functions use a **dictionary**, formed from the database using the database.py file, which uses fund names as keys and **tuples** with other information as values. The results are displayed as formatted strings.

Option 2 (Display the Top Performing Funds): This option calls the **top_performing_funds** function to identify and display the best-performing hedge funds based on annual returns. The **sorted_returns** function is used by the **top_performing_funds**, employing the recursive Quicksort algorithm to return a **sorted array** of returns. The results are rearranged in a descending order, and a **new dictionary top_funds**, is created with n funds, where n is specified by the user. The program then displays the name and rank of each of the top n funds for the year 2020.

Option 3 (Display Funds with Positive or Negative Returns): This option calls the **funds_in_profit_or_loss** function to segregate the **dictionary** of hedge funds into those with positive and negative returns. The function produces a **tuple** with two lists, where one list contains funds with positive returns and the other with negative returns. Each list includes **tuples** where the first element is the fund name and the second is the return. The user can choose to view either positive or negative returning funds.

Option 4 (Calculate Returned Assets): This option calls the **returned_assets** function, which computes the total assets returned by each hedge fund. The returned assets are determined by multiplying each fund's total assets under management by its annual return. The resulting values are stored in **an array**. The user specifies n funds they wish to see, and the program displays the names and returned asset values for the top n funds.

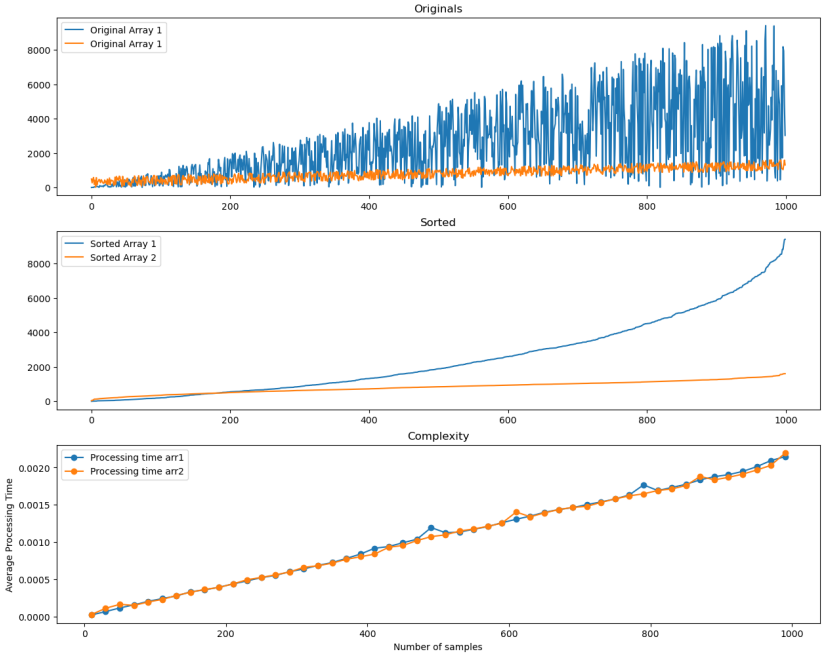
Option 5 (Exit the Programme).

Part 2 - Discussion

Based on the Big-O analysis, the worst-case scenario for Quicksort is O(N^2), which occurs when unbalanced partitions are created and O($Nlog(N)$) for the best/average case. Under these conditions, it was expected that the processing time could show quadratic growth as the sample size increased. As a comparison algorithm, Bubblesort algorithm with the same worst case scenario and a better O(N) best case scenario is used in this text.

In the plots of the original arrays, the first generated random array demonstrates significant fluctuations, while the second one follows a more uniform distribution, due to the given conditions of functions determining the array generation. After applying Quicksort, both arrays were sorted

successfully, confirming the Quicksort's proper execution. In the complexity plot, the processing time for both arrays (slighlty worse for the array 1, possibly since the larger difference between element can sometimes cause a larger imbalance when creating partitions) increases linearly (with slight fluctuations) with input size, deviating from the anticipated quadratic growth associated with the worst case scenario. Thus, it can be said that the commonly used pivot (the last element of the array), seems like the right choice and the algorithm followed the best/average case scenario. Therefore, to compare Quicksort and Bubblesort, it may be appropriate to compare the times for the best cases: O($Nlog(N)$) and O(N). Since $N(>1)$ is always smaller than $Nlog(N)$, Bubblesort seems more efficient. However, due to the Divide and Conquer technique, which can also be implemented by several computers, making it even faster, larger datasets may not allow the best case scnario for Bubblesort to occur so often. Since the worst case scenario is the same for both algoritms, it can be concluded that Quicksort is more efficient for larger datasets, while the opposite holds for Bubblesort.



Acknowledgements

I have not used any additional advice from outside sources while working on this project. However, I have used a part of the dataset other than one of those provided. The dataset used can be found by using the following link: https://www.kaggle.com/datasets/ironbatshashank/top-100-global-hedge-funds-2020 ("Top 100 Global Hedge Funds 2020" by Shashank Srivastava).

Authorship declaration

> I claim that this assignment is my own work. I have not used any AI or help not authorised by the course. My work respects the university and course code of conduct.

* The entire database was not utilized because, for the purposes of this project, the marginal benefit of including more entries seemed to be negligible.